Repository Structure Revision v2.0

```
treasury-label-verifier/

README.md
LICENSE
.gitignore
requirements.txt
app.py

docs/
    project-charter.md
    project-design-document.md
    technical-planning-document.md
    architecture.md
    assumptions-and-limitations.md
    test-plan.md

ocr/
    ocr_service.py
    image_preprocessing.py

matching/
    record_matcher.py
    scoring.py

validation/
    validator.py
    field_extractor.py
    warning_rules.py

utils/
    text_normalizer.py
    csv_loader.py
    timer.py
    export_results.py

sample-labels/
    valid/
    invalid/
    poor-quality/
```

sample-data/
  valid-batch.csv
  mixed-errors-batch.csv
  missing-warning-batch.csv
  record-matching-demo.csv

sample-batches/
  clean-batch/
  mixed-error-batch/
  poor-quality-batch/

deployment/
  render.yaml
  deployment-notes.md

Purpose of Key Files and Folders

README.md

Primary project overview for reviewers.

Includes:

- Project purpose
- Setup instructions
- Deployment link
- Design decisions
- Assumptions
- Limitations
- Future improvements

---

requirements.txt

Python dependency list.

Expected dependencies include:

- streamlit

- pandas
- pillow
- pytesseract
- rapidfuzz
- opencv-python, if image preprocessing is used

---

app.py

Primary Streamlit application entry point.

Responsible for:

- Rendering the user interface
- Managing file uploads
- Calling OCR, matching, and validation modules
- Displaying results
- Supporting CSV export

Documentation Folder

docs/

Stores supporting project documentation.

project-charter.md

High-level mission, principles, scope, non-goals, and definition of success.

project-design-document.md

Product workflow, user types, validation scope, and design philosophy.

technical-planning-document.md

Architecture, technology stack, modules, and technical decisions.

architecture.md

System architecture diagram and module responsibilities.

assumptions-and-limitations.md

Clear explanation of assumptions, prototype boundaries, and deferred production features.

test-plan.md

Test cases for valid labels, invalid labels, poor-quality images, record matching, and batch processing.

OCR Folder

ocr/

Contains OCR-related functionality.

ocr_service.py

Responsible for:

- Receiving image input
- Running Tesseract OCR
- Returning extracted raw text

image_preprocessing.py

Optional helper functions for:

- Basic image cleanup
- Contrast adjustment
- Grayscale conversion
- OCR preparation

Image preprocessing is not a core requirement but may improve OCR accuracy if time permits.

Matching Folder

matching/

Contains record-matching functionality.

record_matcher.py

Responsible for:

- Comparing OCR-extracted label text against CSV application records
- Identifying the most likely record
- Returning match status and confidence

scoring.py

Responsible for:

- Applying match scoring rules
- Combining brand, class/type, ABV, and net contents scores
- Applying confidence thresholds

Confidence thresholds:

95% and above → Auto Match
70%–94% → Needs Review
Below 70% → No Match

Validation Folder

validation/

Contains field extraction and validation logic.

validator.py

Responsible for:

- Applying field-level validation rules
- Returning Pass, Fail, Missing, or Needs Review

field_extractor.py

Responsible for:

- Identifying likely field values from OCR text
- Extracting probable label information

warning_rules.py

Responsible for:

- Government warning statement validation
- Strict warning text checks
- Warning capitalization checks

Utility Folder

utils/

Contains shared helper logic.

text_normalizer.py

Purpose:

Improve OCR reliability before matching and validation.

Examples:

- Normalize casing
- Clean whitespace
- Standardize units
- Reduce common OCR formatting issues

---

csv_loader.py

Purpose:

Load and validate application CSV records.

Required CSV fields:

- record_id
- brand_name
- class_type
- alcohol_content
- net_contents
- government_warning

Optional CSV fields:

- bottler_producer
- country_of_origin
- beverage_type

If required fields are missing, processing should stop and the user should receive a plain-English error message.

---

timer.py

Purpose:

Track:

- Per-label processing time
- Total batch processing time

---

export_results.py

Purpose:

Generate downloadable CSV results.

Exported results may include:

- Image name
- Matched record ID
- Matched brand name
- Match confidence
- Overall status
- Field-level results
- Processing time
- Notes

Sample Labels Folder

sample-labels/

Contains individual sample label images.

valid/

Labels expected to pass validation.

invalid/

Labels with known validation problems.

Examples:

- Brand mismatch
- ABV mismatch
- Net contents mismatch
- Government warning mismatch

poor-quality/

Labels with OCR challenges.

Examples:

- Blur
- Low contrast
- Glare
- Cropping

Poor-quality examples should generally route to Needs Review rather than automatic Fail.

Sample Data Folder

sample-data/

Contains application record CSV files.

valid-batch.csv

Clean expected records for passing examples.

mixed-errors-batch.csv

Records designed to test mismatches and review flags.

missing-warning-batch.csv

Records designed to test government warning validation.

record-matching-demo.csv

Records designed to test image-to-record matching where image filenames are not relied upon.

Sample Batches Folder

sample-batches/

Contains grouped image batches for reviewer testing.

clean-batch/

A small batch of labels expected to pass.

mixed-error-batch/

A batch containing both passing labels and labels with known issues.

poor-quality-batch/

A batch containing labels that should trigger Needs Review due to OCR uncertainty.

Deployment Folder

deployment/

Contains deployment configuration and notes.

render.yaml

Optional Render deployment configuration.

deployment-notes.md

Documents:

- Deployment platform
- Environment setup
- Public URL
- Known deployment limitations
- Tesseract dependency notes

Removed From Earlier Architecture

The previous structure included:

frontend/
backend/
api/

These folders are no longer part of the recommended architecture.

Reason:

The final prototype uses Streamlit, which combines the user interface and application workflow into a single Python application.

This reduces complexity, avoids frontend/backend integration overhead, and better fits the prototype timeline.

CSV Export

Status:

Core workflow support

Purpose:

Allow reviewers to download verification results for follow-up review, supervisor review, or documentation.

Format:

CSV

Exports may include:

- Image Name
- Matched Record ID
- Overall Status
- Field Results
- Match Confidence
- Processing Time
- Reviewer Notes

Future Production Structure Considerations

A production version may eventually separate the application into:

- Frontend
- Backend API
- Database
- Authentication layer
- Audit logging service
- Queue-based batch processor
- COLAs Online integration layer

Those elements are intentionally excluded from the prototype to avoid unnecessary complexity.